

Traducción Automática: EUROPARL

Miguel García Bohigues

Enero 2023

1 Introducción

La traducción automática es un problema abierto y candente en el mundo del aprendizaje automático. Durante mucho tiempo, los modelos basados en la estadística han reinado en esta disciplina, pero con la llegada de los transformer y los modelos de atención en 2017 parece que han quedado un poco subyugados. En el presente proyecto nos planteamos el objetivo de desarrollar dos modelos de traducción automática, uno estadístico y otro neuronal, para comparar su rendimiento a la hora de generar traducciones sobre un mismo corpus: *EuroParl*.

Para generar el modelo de traducción estadístico, vamos a utilizar *Moses*. *Moses* es un sistema que permite entrenar modelos de traducción automática para cualquier par de lenguas de entrada y de salida. Dados un conjunto de ficheros pares de entrada y salida, *Moses* entrena un modelo sobre el que luego proporciona un algoritmo de búsqueda que permite encontrar rápidamente la traducción de máxima probabilidad. Es una herramienta cuya última versión fue lanzada en octubre del 2017. Para un software actual este es un periodo de tiempo muy eveladado sin disponer de una nueva versión. No obstante dispone tanto de repositorio de git, como de un manual que ayuda a su uso e instalación. Como softwares complementarios, se ha utilizado también *Giza* y *SRILM*.

Para el desarrollo del modelo neuronal vamos a utilizar la herramienta *OpenNMT*. *OpenNMT*, al igual que *Moses*, permnite entrenar modelos de traducción a partir de ficheros de pares de lenguas. De entre todas las funcionalidades que ofrece el toolkit, vamos a centrarnos en los modelos de transformer.

Cómo metrica común para evaluar las traducciones producidas por ambos sistemas, utilizaremos el *BLEU*, un método de evaluación que presenta un valor más elevado cuanto más similar es la frase producida con respecto a otra de referencia.

2 Corpus

El corpus sobre el que vamos a trabajar y desarrollar los modelos de traducción va a ser *EuroParl*. Éste es un conjunto de textos extraídos de los procedimientos del parlamento europeo traducidos a 21 lenguas de los países del continente. En concreto, hemos trabajado con la versión número 7, que presenta un mayor número de muestras que su precesora.

Concretamente, nuestro corpus es un subconjunto del original que consta de 35000 frases en la partición de entrenamiento y de 1000 para validación. Para poder ajustar los pesos del modelo, hemos separado el conjunto de entrenamiento en dos: uno de entrenamiento y otro de desarrollo.

No obstante, para la fase de experimentación, debido al elevado tamaño del corpus y la limitación computacional y temporal, hemos utilizado un subconjunto más pequeño del corpus. En la tabla 1 se encuentra la información de las dimensiones, tanto del corpus original como del reducido.

Como vemos, el corpus reducido representa aproximadamente entre un 15% y un 20 % del corpus original. de esta manera, podemos agilizar la fase de

	EuroParl	MINI-EuroParl
Train	37,5K	3,75K
Dev	12,5K	1,875K
Test	1K	0,2K

Table 1: Caption

experimentación y obtener más rápidamente la combinación de parámetros que maximiza el *BLEU*.

3 Experimentación

La fase de experimentación la vamos a separar en dos secciones: una para la experimentación sobre *MOSES* y otra sobre *Open-NMT*, debido a que los parámetros a estimar y el procedimiento a seguir es completamente diferente.

Un apunte importante antes de comentar el apartado de experimentaciones, el hecho de haber trabajado con un subconjunto tan pequeño ha podido hacer que haya palabras del test que hayan quedado fuera del vocabulario y que su representación sea ‘UKN’, por ello los resultados que vamos a ver en la experimentación van muy inferiores al resultado del modelo final.

3.1 MOSES

Para el caso de *MOSES*, los parámetros que podemos variar en el modelo son:

- El tamaño del N-grama
- El número máximo de iteraciones
- El suavizado a utilizar
- El método de descuento
- El método de alineamiento de palabras

Siguiendo los parámetros por orden, hemos buscado el mejor tamaño de N-gramas para el problema. Con el conocimiento obtenido sobre *MOSES*, hemos acotado los valores a los que se ven en la tabla 2. En ella también podemos ver cómo el tamaño que mejor resultado ha reflejado ha sido 4. Por lo tanto, de ahora en adelante será este el tamaño que escojamos para realizar el resto de pruebas.

En la evaluación previa que realizamos con *MOSES* y EuTrans, vimos que modificar el número máximo de iteraciones no arrojaba variaciones significativas. Por ello para este experimento hemos probado con 5 y 10 iteraciones máximas exclusivamente. El *BLEU* obtenido ha sido de 11.43 y 11.54 respectivamente, pero no podemos decir que una configuración sea mejor que la otra, puesto que estos resultados pueden verse afectados por el tokenizador.

Ngram	BLEU
3	11.26
4	11.53
5	11.35

Table 2: Bleu en función del tamaño de n-grama

Los siguientes hiperparámetros a estimar son el método de descuento y el suavizado. Hemos decidido estimarlos en conjunto combinándolos por pares. En concreto hemos probado combinaciones de descuentos de Knesser-ney, Knesser-ney sin modificar y Witten Bell con ambos backoff e interpolación. En la tabla 3 se muestran los resultados obtenidos.

Suavizado	Descuento	BLEU
interpolation	un.Knesser-ney	11.12
interpolation	Knesser-ney	11.65
interpolation	Witten-bell	11.25
backoff	un.Knesser-ney	11.79
backoff	Knesser-ney	11.41
backoff	witten-bell	11.46

Table 3: bleu en función del suavizado y el método de descuento

Como vemos, los resultados han sido un tanto dispares, fruto posiblemente de la reducida dimensionalidad del corpus. No obstante vemos cómo el método de descuento de Knesser-ney sin modificar y con suavizado por backoff es la combinación que mejor resultado refleja. A pesar de ello, con el conocimiento que disponemos, creemos que interpolación debería funcionar mejor, por lo que aplazamos la decisión sobre el suavizado a futuros experimentos.

Lo siguiente a acordar, es el método de alineamiento a utilizar en el modelo resultante. Hemos experimentado con 3 diferentes: grow-diag, grow-diag-final y grow-diag-final-and. En la tabla 4 se muestran los resultados obtenidos.

Suavizado	Alineamiento	BLEU
backoff	grow-diag	11.44
backoff	grow-diag-final	10.88
backoff	grow-diag-final-and	11.37
interpolation	grow-diag-final-and	11.69

Table 4: Bleu en función del alineamiento

Los experimentos reflejan una mejoría en el BLEU con la utilización del alineamiento *grow-diag-final-and*. Además, vemos cómo para éste, el suavizado por interpolación ha devuelto mejor resultado que el suavizado por backoff.

Por lo tanto, nuestro modelo final queda con la siguiente configuración de hiperparámetros:

- Tamaño cuatro de n-grama
- 10 iteraciones como máximo
- Suavizado por interpolación
- Descuento de Kneseer-ney sin modificar
- Alineamiento de palabras *grow-diag-final-and*

Con esta combinación hemos corrido el experimento con todo el corpus, esta vez tokenizado, y ese resultado ha sido de $BLEU = 27.05, 59.7/32.7/20.5/13.4$ (BP=1.000, ratio=1.011, hyp_len=30398, ref_len=30082)

3.2 Open-NMT

Para el caso de *Open-NMT*, los parámetros con los que hemos intentado ajustar el modelo han sido:

- El tamaño del embedding.
- La dimensión de la capa de Feed-Forward
- El número de capas
- El tamaño del batch
- Y por último el optimizador utilizado.

Las pruebas, en este caso, se han desarrollado sobre el corpus MINI-Europarl, introducido anteriormente. El primer experimento que hemos realizado ha sido con el tamaño del embedding. Partiendo del conocimiento adquirido realizando las prácticas, hemos probado tamaños de 128, 256, 512 y 1024. En la tabla 5 se muestran los resultados que hemos obtenido.

w-enc-size	BLEU
128	6.5
256	7.0
512	8.0
1024	6.2

Table 5: Bleu en función del tamaño del embedding

Como se puede apreciar, a medida que vamos aumentando el tamaño del embedding, vamos consiguiendo mejor resultado del *BLEU*, hasta llegar a 1024, quedando 512 como punto de convergencia y valor que maximiza el *BLEU*.

Todo seguido, con el valor del embedding ya decidido, pasamos a estimar el mejor valor de tamaño para la capa Feed-Forward. Para este caso hemos partido ya de la configuración con mejor tamaño del embedding. Tal y como reflejan los experimentos mostrados en la tabla 6, hemos partido del valor por

ff-size	BLEU
64	8.0
128	8.2
256	7.7

Table 6: Bleu en función del tamaño de la capa FF

defecto incrementando en potencias de dos el valor. Tras 3 incrementos ya hemos encontrado el punto de convergencia, siendo este 128.

Todo seguido hemos utilizado esta configuración para estimar el número idóneo de capas. En este caso no hemos encontrado ninguna configuración que mejore el valor por defecto, tal y como demuestra la tabla 7

n-layers	BLEU
1	7.3
2	8.2
3	6.8
4	6.7

Table 7: Bleu en función del número de capas

Con los hiperparámetros anteriores, hemos procedido a comprobar el funcionamiento de diferentes optimizadores aplicados a la tarea. En concreto, hemos probado Adam, con $\beta_1 = 0.9$, $\beta_2 = 0.998$ y *decay noam*; SGD con un factor de aprendizaje de 1; y Agadrad con un acumulador inicial de 0.15 y un factor de aprendizaje de 0.1. En la tabla 8 se pueden observar los resultados.

Optimizer	BLEU
Adam	8.2
SGD	7.6
Adagrad	8.0

Table 8: Bleu en función del optimizador utilizado

Como vemos, ninguno de los optimizadores ha conseguido superar el resultado que ofrece Adam con la configuración especificada previamente. Además, vemos como Adagrad, el los parámetros anteriores ha conseguido superar a SGD.

Finalmente, hemos ajustado el resultado variando el tamaño del batch y quedándonos con el mejor valor obtenido. Tal y como refleja la tabla 9, hemos probado con tamaños entre 64 y 256, esta vez ya sobre el corpus completo y con todos los hiperparámetros anteriores.

Batch-size	BLEU
64	20.0
128	22.5
256	21.2

Table 9: Bleu en función del tamaño de batch tras 35000 iteraciones

Con todos los experimentos reflejados, nuestro modelo final se compone de :

- 512 como tamaño del embedding
- 128 neuronas en la FF
- 2 capas
- Adam como optimizador
- Y un tamaño de batch de 128

Con esta combinación el mejor resultado que hemos obtenido después de 35000 iteraciones ha sido de 22.5 de BLEU. Cabe destacar que ha habido un estancamiento severo en el entrenamiento de este modelo, siendo que desde el modelo generado en el paso 27000 el porcentaje de acierto en sobre el conjunto de entrenamiento se ha mantenido al rededor del 62%, siendo incapaz de mejorar, llegando a obtener peores resultados en validación.

4 Ampliaciones

4.1 GRU

Como una de las posibles ampliaciones, hemos configurado y entrenado un modelo de lenguaje basado en GRU aplicado al corpus EUROPARL, con el que estamos trabajando.

Para llevarlo a cabo hemos utilizado, al igual que con el Transformer, el toolkit de *Open-NMT*. Con éste, hemos modificado la configuración para hacer uso de redes neuronales recurrentes de tipo GRU. Hemos establecido un tamaño de embedding y de red de 256 además de utilizar Adam con un factor de aprendizaje de 0.001. Un fragmento de la configuración utilizada se puede ver a continuación.

```

1 # Data
2 save_data: drive/MyDrive/NMT/dataset/EuroParl
3 src_vocab: drive/MyDrive/NMT/dataset/EuroParl/EuroParl-en.vocab
4 tgt_vocab: drive/MyDrive/NMT/dataset/EuroParl/EuroParl-es.vocab
5 overwrite: False
6
7 # Corpora:
8 data:
9 ...
10 # Model

```

```

11 decoder_type: rnn
12 encoder_type: rnn
13 word_vec_size: 256
14 rnn_size: 256
15 rnn_type: GRU
16 input_feed: 1
17 layers: 2
18 accum_count: 8
19 optim: adam
20 learning_rate: 0.001
21 max_grad_norm: 5.0
22 batch_size: 128
23 batch_type: tokens
24 normalization: tokens
25 dropout: 0.3
26 label_smoothing: 0.1
27 max_generator_batches: 2
28 position_encoding: 'true'
29 ...

```

Los resultados, con este modelo, han sido realmente bajos. Hemos procedido como de costumbre, tokenizando y limpiando los datos antes de pasarlos al modelo. Hemos establecido 20000 iteraciones para el entrenamiento del modelo, pero durante este proceso, hemos visto cómo a partir de la iteración 15000 el valor de precisión sobre el conjunto de entrenamiento se ha estancado en menos del 40%. Esto se ha traducido en que sobre el conjunto de test ha obtenido un valor de BLEU de tan sólo 7.2 puntos.

```

1 {
2   "name": "BLEU",
3   "score": 7.2,
4   "signature": "nrefs:1|case:mixed|eff:no|tok:13a|smooth:exp|version
5     :2.3.1",
6   "verbose_score": "32.8/9.5/4.1/2.1 (BP = 1.000 ratio = 1.146
7     hyp_len = 26154 ref_len = 22820)",
8   "nrefs": "1",
9   "case": "mixed",
10  "eff": "no",
11  "tok": "13a",
12  "smooth": "exp",
13  "version": "2.3.1"
14 }

```

Viendo que el resultado ha sido muy inferior a la traducción estadística y neuronal basada en transformers, hemos aumentado el tamaño del embedding y de la red recurrente a 512. Con esto hemos vuelto a lanzar 20000 iteraciones y, en este caso, hemos visto como el estancamiento ha llegado más pronto. En la iteración 8000 la precisión sobre el conjunto de test era del rango del 33% y la perplejidad del rango de 37. Tras 4000 iteraciones, en el paso 12000, la precisión había subido a 34 y la perplejidad había bajado a 36.93. A partir de este punto, el sistema ha mantenido esta oscilación mejorando y empeorando estos dos resultados hasta que en la iteración 18000 ha empezado a decrementar su calidad de manera notable.

Haciendo recapitulación en el entrenamiento, hemos escogido el mejor mod-

elo de todos los generados y hemos generado la traducción con él. Los resultados han demostrado ser peor que en el caso anterior, mostrando un BLEU de 6.7.

4.2 Fairseq

Como segundo desarrollo opcional, hemos seleccionado el toolkit fairseq, mantenido actualmente por Facebook/Meta.

Fairseq es un toolkit neuronal de alto nivel que, al igual que *Open-NMT* proporciona scripts para entrenar modelos de traducción y realizar los procesos de inferencia. Su instalación se puede llevar a cabo a través del clonado del repositorio del proyecto. No obstante tiene una limitación considerable, y es que requiere de una GPU de NVIDIA para poder trabajar.

En nuestro caso, disponemos de una NVIDIA GeForce GTX 1050 Mobile con 2GB de memoria dedicada. Lamentablemente no presenta de una potencia a la altura de las máquinas actuales, por lo que el modelo que hemos probado se ha tenido que adaptar a esta situación.

En concreto hemos escogido una de las configuraciones con las que parte *Fairseq*: “Transformer-tiny” una versión con baja dimensionalidad del modelo transformer. Este modelo dispone tanto de un tamaño de embedding, de feed-forward, de encoder y de decoder de 64. Todo esto con dos cabezas. Con esto, los entrenamientos diponían un coste temporal de 3 minutos y medio por epoch, algo asumible.

Así pues, primero hemos tokenizado los datos con el script que *fairseq* dispone para ello. Y posteriormente hemos entrenado el modelo durante 40 epoch (poco más de 2h). Esto lo hemos hecho con los comandos siguientes:

```
1 fairseq-preprocess --source-lang en --target-lang es --trainpref
  ../Corpus50000/europarl-v7.es-en-train-red.clean --validpref
  ../Corpus50000/europarl-v7.es-en-dev-red.clean --testpref ../
  Corpus50000/europarl-v7.es-en-test.clean --destdir data-bin/
  europarl-v7.en-es
2
3 CUDA_VISIBLE_DEVICES=0 fairseq-train data-bin/europarl-v7.en-es
  --optimizer nag --lr 0.25 --clip-norm 0.1 --dropout 0.2 --max-
  tokens 4000 --arch transformer_tiny --save-dir .
```

Durante el proceso de entrenamiento hemos observado cómo el sistema evolucionaba lenta pero positivamente, teniendo un decremento constante de la perplejidad y un aumento de la precisión. Con el modelo entrenado, hemos procedido a generar las traducciones y computar el BLEU obtenido por las mismas. Lo hemos podido hacer con el siguiente comando:

```
1 fairseq-generate data-bin/europarl-v7.en-es --path ./
  checkpoint40.pt --batch-size 128 --beam 5
```

Con él, hemos ocnseguido un BLEU = 12.88, 44.7/18.6/9.5/4.8 (BP=0.924, ratio=0.927, syslen=21303, reflen=22982), aunque onservando las últimas épocas de entrenamiento, podemos afirmar que el resultado hubiese mejorado de darle más tiempo. No obstante, como éste también era limitado, no hemos probado con más épocas.

5 Conclusiones

En el presente proyecto hemos experimentado con un corpus de gran tamaño y hemos desarrollado y ajustado diferentes modelos de traducción, tanto estadística como neuronal.

Además, hemos podido revisar tanto tecnologías diferentes, en el caso de las LSTM o GRU, así como nuevos toolkit similares al utilizado en las prácticas, como es el caso de Fairseq.

Por desgracia, la limitación de recursos tanto temporales como computacionales de la que disponemos y los elevados tiempos de entrenamiento han limitado bastante los experimentos. No obstante los resultados obtenidos han sido positivos, siendo que consiguen mejorar los proporcionados como baseline (26 para MOSES y 17 para Open-NMT)